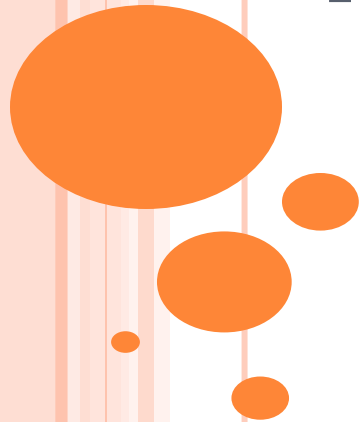


PRINCIPIOS SOBRE INTELIGENCIA ARTIFICIAL GENERATIVA



Inteligencia artificial generativa

- La inteligencia artificial generativa es un modo de inteligencia artificial.
- A partir de una información de entrada, los modelos que la implementan son capaces de generar texto, imágenes, documentos, y contenido en general, tal como lo haría un ser humano.
- Un tipo de modelo basado en IA generativa es el LLM .



LLM

- Un LLM (Large Language Model) es un model de IA generativa basado en lenguaje natural.
 - Los LLMs están especializados en el procesamiento del lenguaje natural para realizar tareas de tipo procesamiento y análisis de textos, resúmenes, organización de datos, generación de código, etc.
 - Una de las implementaciones más conocidas y populares de LLM es GPT
- 

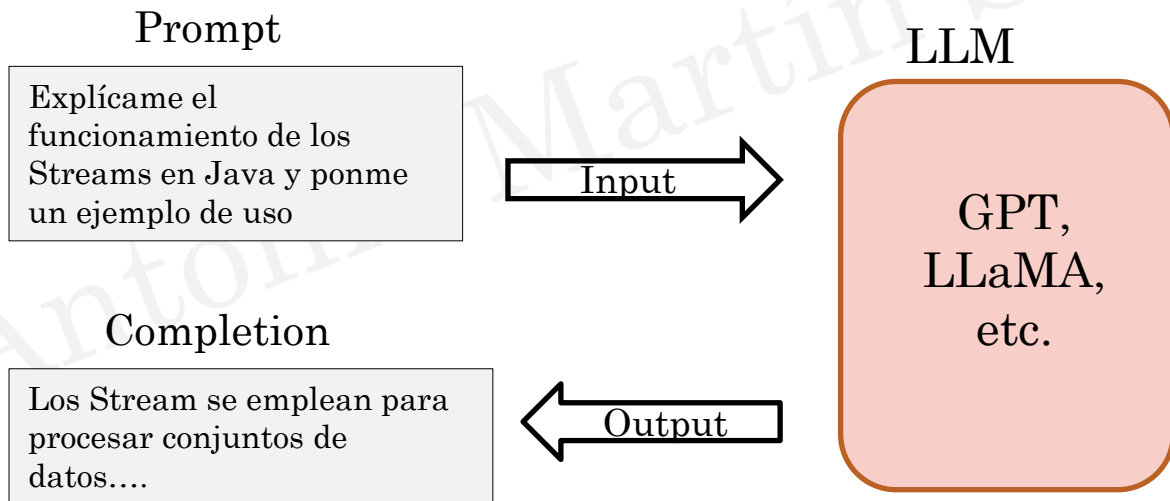
GPT/Chat GPT

- GPT es una implementación de modelo LLM, basado en Transformers y especializado en generación de texto
- Desarrollado por la empresa Open AI
- La aplicación más popular basada en GPT es Chat GPT
- Además de GPT, otros LLM empleados habitualmente: LLaMA, Claude, Gemini, etc.



Nomenclatura

- **Prompt:** Texto en lenguaje natural que se proporciona al LLM con la descripción de lo que tiene que hacer
- **Completion:** Texto resultante generado por el LLM

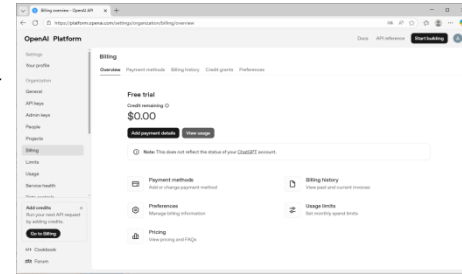
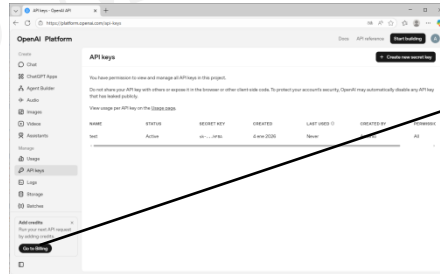
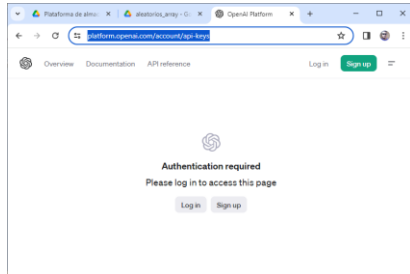


Generación de clave Open AI

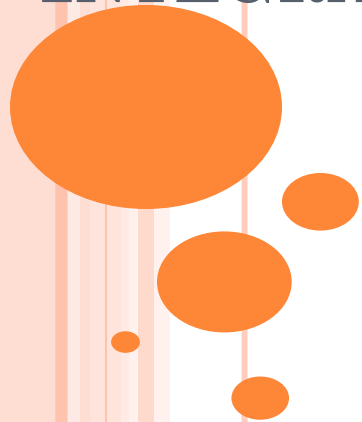
➤ Para poder utilizar GPT necesitamos generar una clave proporcionada por el propio Open AI:

<https://platform.openai.com/account/api-keys>

➤ Una vez generada la clave, debemos asociarla a un método de pago.



INTEGRACIÓN EN APLICACIONES SPRING



Fundamentos y objetivos

- Es posible integrar modelos LLM en aplicaciones Spring
- El objetivo es poder desarrollar aplicaciones en la que se haga un uso personalizado de LLMs, integrando la inteligencia artificial en aplicaciones Java
- Ejemplos: Servicios personalizados de búsqueda de información, chat boots, agentes IA, etc.



GPT en Spring

- Spring Boot proporciona una librería para poder integrar GPT en aplicaciones Spring
- A través de ella podemos enviar prompts personalizados y manipular las respuestas recibidas desde el LLM
- Se requiere el starter OpenAI:

```
<dependency>  
  <groupId>org.springframework.ai</groupId>  
  <artifactId>spring-ai-openai-spring-boot-starter</artifactId>  
</dependency>
```

Configuración API Key

➤ Para poder hacer uso de los componentes de Spring Open AI en una aplicación es necesario registrar el API Key en el .properties o .yaml

```
spring.ai.openai.api-key=sk_*****
```

➤ Opcionalmente, puede indicarse el modelo específico de GPT a utilizar. Por defecto, es GPT-4o-mini

```
spring.ai.openai.chat.options.model=gpt-4o
```



Interfaz ChatClient

- Permite realizar peticiones al LLM.
- Para crear un objeto ChatClient, se utiliza el objeto ChatModel de la librería Open AI, el cual es inyectado automáticamente por Spring:

```
@Bean  
public ChatClient chatClient(ChatModel chatModel) {  
    return ChatClient.builder(chatModel).build();  
}
```



Interacción con el modelo

- El objeto `ChatClient` es inyectado en la capa de lógica de negocio, desde donde se interaccionará con el LLM.
- Encadenando una serie de llamadas a métodos configuramos la petición y obtenemos la respuesta:

```
return chatClient
    .prompt() // Crea objeto ChatClientRequestSpec
    .user(pregunta) // Configura texto de la pregunta. Devuelve objeto del mismo tipo anterior
    .call() // Ejecuta la operación. Devuelve CallResponseSpec
    .content(); // Obtiene el texto de la respuesta
```

Personalización de prompts

➤ Mediante el uso de placeholders ({data}), podemos personalizar un prompt y proporcionar valores en tiempo de ejecución:

```
String prompt="Indicame {num} direcciones Web en las que aprender Python";
```

```
PromptTemplate template=new PromptTemplate(prompt);  
Map<String, Object> variables=new HashMap<>();  
variables.put("num", 5);  
String render=template.render(variables);  
  
return chatClient  
    .prompt()  
    .user(render)  
    .call()  
    .content();
```

Respuesta como objetos

➤ **Utilizando el prompt adecuado, podemos indicarle al LLM que nos entregue el resultado en un formato específico, por ejemplo JSON, y mapearlo a Java:**

```
String prompt="Búscame los libros... y devuelve el resultado como una lista de JSON con las propiedades titulo, autor y precio";
```

```
return chatClient  
    .prompt()  
    .user(prompt)  
    .call()  
    .entity(Libro[].class);
```

